

endianness-v2

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Here's a file that was recovered from a 32-bits system that organized the bytes a weird way. We're not even sure what type of file it is.

Tools Used

- `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `xxd` – View hexadecimal representation of files
 - **CyberChef** – Web-based tool for endianness conversion and image rendering
-

Complete Solution

Step 1: Download and Prepare

Download the file named `challengefile` from the challenge page.

Step 2: Basic File Inspection

First, verify the file type:

```
file challengefile
```

Expected output:

```
challengefile: data
```

The `file` command doesn't recognize the file type – it's corrupted or has a missing/incorrect header.

Step 3: Search for Visible Text

Check if any readable text reveals the flag:

```
strings challengefile | grep -i "picoCTF"
```

Output: (no results)

No flag yet.

Step 4: Examine Metadata

Use `exiftool` to inspect the file's metadata:

```
exiftool challengefile
```

Output:

```
ExifTool Version Number      : 13.50
File Name                    : challengefile
Directory                   : .
File Size                   : 3.4 kB
File Modification Date/Time  : 2026:05:20 16:41:57+02:00
...
Warning                     : Processing JPEG-like data after unknown 1-byte
header
```

Key finding: The warning `Processing JPEG-like data after unknown 1-byte header` suggests that the file contains JPEG data but has an extra byte or endianness issue at the

beginning.

Step 5: Hexdump Analysis

Let's examine the hexadecimal representation of the file header:

```
xxd challengefile | head
```

Output:

```
00000000: e0ff d8ff 464a 1000 0100 4649 0100 0001  ....FJ....FI....
00000010: 0000 0100 4300 dbff 0606 0800 0805 0607  ....C.....
00000020: 0907 0707 0c0a 0809 0b0c 0d14 1219 0c0b  .....
00000030: 1d14 0f13 1d1e 1f1a 201c 1c1a 2027 2e24  .... '.$
00000040: 1c23 2c22 2937 281c 3431 302c 271f 3434  .#,")7(.410,'.44
00000050: 3238 3d39 3433 2e3c 00db ff32 0909 0143  28=943.<...2...C
00000060: 0c0b 0c09 180d 0d18 211c 2132 3232 3232  .....!..!22222
00000070: 3232 3232 3232 3232 3232 3232 3232 3232  2222222222222222
00000080: 3232 3232 3232 3232 3232 3232 3232 3232  2222222222222222
00000090: 3232 3232 3232 3232 3232 3232 c0ff 3232  222222222222..22
```

Step 6: Identify the Correct File Format

Looking at the hexdump, we see a pattern: `ff d8 ff` – this is the magic signature for a **JPEG** file (`FF D8 FF`). However, the bytes appear to be in the wrong order (endianness issue).

A valid JPEG file should start with `FF D8 FF E0` or similar. Here, we have `E0 FF D8 FF` – the bytes are swapped (32-bit little-endian).

Step 7: Understand the Endianness Problem

The description says the file came from a **32-bit system that organized bytes in a weird way** – this is an endianness issue. On a little-endian system, multi-byte values are stored with the least significant byte first.

For a JPEG file, the correct header should be:

- Byte 0: FF
- Byte 1: D8
- Byte 2: FF
- Byte 3: E0

But in our file, we have:

- E0 FF D8 FF

This suggests that 32-bit words need to be byte-swapped.

Step 8: Fix with CyberChef (Recommended Method)

The easiest way to fix endianness issues is using **CyberChef**:

1. Go to <https://gchq.github.io/CyberChef/>
2. Upload the challengefile
3. Add the following operations in order:

Operation	Settings
To Hex	No additional settings
Swap endianness	Word length: 4 bytes
Render Image	Input format: Hex, Output format: Image

Download CyberChef

Last build: An hour ago

Options

About / Support

Operations

render

Render Image

Render Markdown

Fang URL

HTML To Text

Hex Density chart

Template

To Table

Favourites

Data format

Encryption / Encoding

Public Key

Arithmetic / Logic

Recipe

To Hex

Swap endianness

Render Image

STEP

BAKE!

Auto Bake

Input

Output

File details

The image will render directly in CyberChef, revealing the flag!

Step 9: Alternative – Fix with hexedit (Command Line)

If you prefer the command line, you can also fix the file manually:

Create a script to swap 4-byte words:

```
# Read the file, swap each 4-byte word, and write to new file
hexdump -v -e '1/4 "%08x"' -e '"\n"' challengefile | while read line; do
    echo -n "$line" | sed 's/\(..\)..\(..\)..\(..\)..\(..\)/\4\3\2\1/' | xxd -r -p
done > fixed.jpg
```

Or use a simpler one-liner:

```
hexdump -v -e '1/4 "%08x"' -e '"\n"' challengefile | sed 's/\(..\)..\(..\)..\(..\)..\(..\)/\4\3\2\1/' | xxd -r -p > fixed.jpg
```

Step 10: Verify the Repaired File

Check if the file is now recognized:

```
file fixed.jpg
```

Expected output:

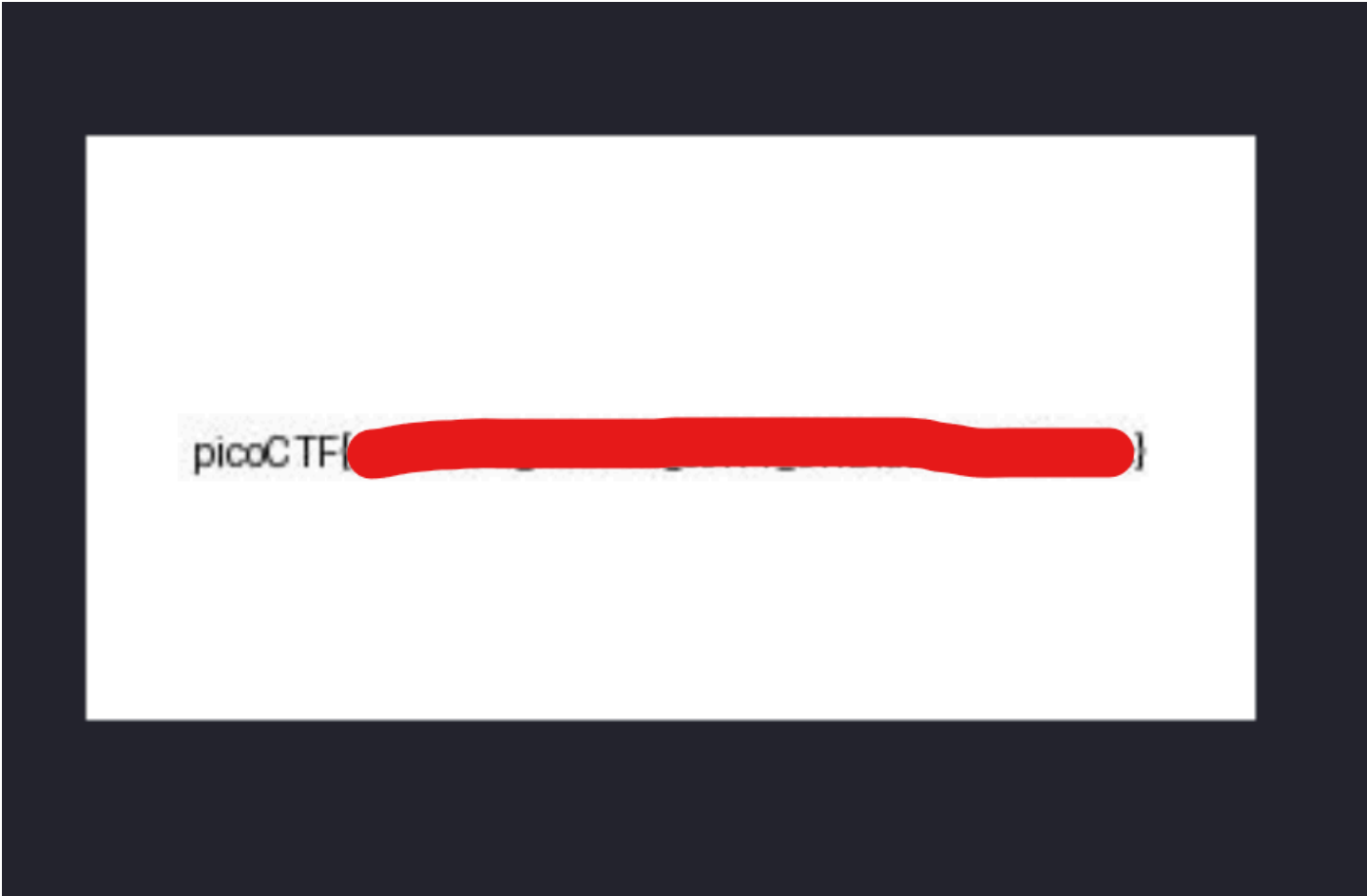
```
fixed.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, segment
length 16, baseline, precision 8, 300x150, components 3
```

Success! The file is now a valid JPEG image.

Step 11: Open the Image

Open the repaired image:

```
xdg-open fixed.jpg    # Linux
open fixed.jpg        # macOS
start fixed.jpg       # Windows
```



The flag is visible directly in the image!

Key Concepts

Concept	Description
Endianness	The order in which bytes are arranged within multi-byte values (little-endian vs big-endian).
Little-Endian	Least significant byte is stored first (used by x86/32-bit systems).
Big-Endian	Most significant byte is stored first (used by network protocols and many file formats).

Concept	Description
JPEG Magic Bytes	JPEG files start with <code>FF D8 FF</code> (SOI marker).
Byte Swapping	The process of reversing the order of bytes to convert between endianness.
CyberChef	A powerful web-based tool for data transformation, including endianness conversion and image rendering.

Pro Tips

- Look for common file signatures** – Even when bytes are swapped, recognizable patterns like `FF D8` (JPEG) or `89 50` (PNG) can appear in the hexdump.
- 32-bit systems often use little-endian** – The description mentioning a “32-bits system” suggests bytes may need to be swapped in 4-byte chunks.
- CyberChef is your best friend for endianness** – It has built-in operations for “Swap endianness” and can render images directly.
- Test the hexdump** – If you see reversed patterns like `E0 FF D8 FF` instead of `FF D8 FF E0` , try swapping bytes.
- 4-byte swapping is common** – Use `hexdump -e '1/4 "%08x"'` to read the file as 4-byte words, then reverse each word.
- Don't forget to verify with `file`** – After fixing, always check that the file type is recognized correctly.